Lab 07 — PKI: Build Your Own Certificate Authority

> **SY0-701 Objective 1.4** — Explain the importance of using appropriate cryptographic solutions.
>
> **Domain 1** — General Security Concepts (12%) · **Time:** ~35 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Run the full certificate lifecycle on a private CA: create a root of trust, generate a wildcard CSR, issue the certificate, trust the root, check status with OCSP, revoke it, publish a CRL, and compare self-signed and third-party certificates.

You'll build: A root CA (`/root/ca`), a CA-signed wildcard certificate for `*.lab.local` with SAN, AIA and CRL extensions, a running OCSP responder that reports `good` and then `revoked`, a CRL listing the revoked serial, and a self-signed certificate for comparison.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 06 (public and private keys) is helpful; tools installed in step 1.

Background

- A **certificate authority (CA)** verifies an identity and signs a certificate that binds a **public key** to a subject. The CA's own root certificate is the **root of trust**.
- A **certificate signing request (CSR)** contains the requester's public key and subject details; the **private key never leaves the requester**.
- A **wildcard** certificate (`*.example.com`) covers every first-level subdomain; **Subject Alternative Name (SAN)** is the field clients actually check.
- **Revocation**: a **CRL** is a signed list of revoked serial numbers published periodically; **OCSP** answers "good, revoked or unknown" for one certificate in real time, and OCSP stapling lets the server deliver the answer.
- **Self-signed** certificates are signed by their own key and are trusted by no one by default; **third-party** CAs are pre-installed in operating system and browser trust stores. **Key escrow** keeps a recoverable copy of keys with a trusted party.

Steps

1. Install the tools and build the CA layout

A CA needs a private key directory, a database of issued certificates (`index.txt`), serial counters and a configuration that defines the issuing policy and the extensions it stamps on certificates.

```
apt-get update -qq && apt-get install -y openssl ca-certificates curl
mkdir -p /root/ca/{certs,private,newcerts,crl} && cd /root/ca
chmod 700 private && touch index.txt && echo 1000 > serial && echo 1000 > crlnumber
cat > /root/ca/openssl.cnf <<'EOF'
[ ca ]
default_ca = CA_default
```

```
[ CA_default ]
dir              = /root/ca
database         = $dir/index.txt
serial           = $dir/serial
new_certs_dir    = $dir/newcerts
certificate      = $dir/certs/ca.crt
private_key      = $dir/private/ca.key
crlnumber        = $dir/crlnumber
default_md       = sha256
default_days     = 397
default_crl_days = 7
policy           = pol_loose
copy_extensions  = copy
unique_subject   = no
x509_extensions  = server_cert
[ pol_loose ]
commonName       = supplied
organizationName = optional
[ req ]
distinguished_name = req_dn
[ req_dn ]
[ v3_root ]
basicConstraints       = critical, CA:TRUE
keyUsage               = critical, keyCertSign, cRLSign
subjectKeyIdentifier   = hash
[ server_cert ]
basicConstraints       = critical, CA:FALSE
keyUsage               = critical, digitalSignature, keyEncipherment
extendedKeyUsage       = serverAuth
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid, issuer
authorityInfoAccess    = OCSP;URI:http://127.0.0.1:2560
crlDistributionPoints  = URI:http://127.0.0.1/lab.crl
EOF
ls /root/ca
```

2. Create the root CA (the root of trust)

Generate a 4096-bit root key and a self-signed root certificate valid for ten years. Subject equals issuer, and `CA:TRUE` allows it to sign other certificates. In production this key lives offline or in an HSM.

```
cd /root/ca
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out private/ca.key
2>/dev/null && chmod 600 private/ca.key
openssl req -x509 -new -key private/ca.key -sha256 -days 3650 -config openssl.cnf -
extensions v3_root -subj "/CN=Lab Root CA/O=Tertiary Infotech Lab" -out certs/ca.crt
openssl x509 -in certs/ca.crt -noout -subject -issuer -dates
openssl x509 -in certs/ca.crt -noout -ext basicConstraints,keyUsage
```

3. Generate a key and a wildcard CSR

The web team creates its own key pair and a CSR for `*.lab.local`. The CSR carries the public key, subject and requested SANs, and is self-signed to prove possession of the private key.

```
cd /root/ca
openssl req -new -newkey rsa:2048 -nodes -keyout private/web.key -subj
"/CN=*.lab.local/O=Lab Web Team" -addext "subjectAltName=DNS:*.lab.local,DNS:lab.local" -
out web.csr 2>/dev/null
openssl req -in web.csr -noout -verify -subject
openssl req -in web.csr -noout -text | grep -A1 "Subject Alternative Name"
```

4. Sign the CSR as the CA

The CA checks the request against its policy, copies the SAN extension, adds its own server extensions (EKU, OCSP and CRL locations) and signs the certificate.

```
cd /root/ca
openssl ca -batch -config openssl.cnf -in web.csr -out certs/web.crt 2>&1 | tail -2
openssl x509 -in certs/web.crt -noout -subject -issuer -serial
openssl x509 -in certs/web.crt -noout -ext
subjectAltName,extendedKeyUsage,authorityInfoAccess,crlDistributionPoints
cat index.txt
```

`index.txt` now has a `V` (valid) line for serial `1000`.

5. Verify the chain and add the root to the trust store

The new certificate is not trusted by default because the system trust store contains only third-party roots. Verify against your CA, then install the root as a trust anchor.

```
cd /root/ca
openssl verify certs/web.crt 2>&1 | tail -2
openssl verify -CAfile certs/ca.crt certs/web.crt
cp certs/ca.crt /usr/local/share/ca-certificates/lab-root-ca.crt
update-ca-certificates 2>&1 | grep added
openssl verify certs/web.crt
```

```
error 20 at 0 depth lookup: unable to get local issuer certificate
error certs/web.crt: verification failed
certs/web.crt: OK
1 added, 0 removed; done.
certs/web.crt: OK
```

6. Check certificate status with OCSP

Start an OCSP responder for the CA, then ask it about the web certificate, exactly as a browser would using the URL from the AIA extension.

```
cd /root/ca
nohup openssl ocsp -index index.txt -port 2560 -rsigner certs/ca.crt -rkey private/ca.key
-CA certs/ca.crt -nmin 5 > /root/ca/ocsp.log 2>&1 &
sleep 2
openssl ocsp -issuer certs/ca.crt -cert certs/web.crt -url http://127.0.0.1:2560 -CAfile
certs/ca.crt 2>&1 | grep -E "Response verify|web.crt"
```

```
Response verify OK
certs/web.crt: good
```

7. Revoke the certificate and publish a CRL

The web server's private key has leaked. Revoke the certificate with a reason code, publish a fresh CRL, and confirm that both CRL checking and OCSP now reject it.

```
cd /root/ca
openssl ca -config openssl.cnf -revoke certs/web.crt -crl_reason keyCompromise 2>&1 | tail
-1
openssl ca -config openssl.cnf -gencrl -out crl/lab.crl 2>/dev/null
openssl crl -in crl/lab.crl -noout -text | grep -E -A3 "Revoked Certificates"
openssl verify -crl_check -CAfile certs/ca.crt -CRLfile crl/lab.crl certs/web.crt 2>&1 |
tail -2
pkill -f "openssl ocsp -index" ; sleep 1
nohup openssl ocsp -index index.txt -port 2560 -rsigner certs/ca.crt -rkey private/ca.key
-CA certs/ca.crt -nmin 5 > /root/ca/ocsp.log 2>&1 &
sleep 2
openssl ocsp -issuer certs/ca.crt -cert certs/web.crt -url http://127.0.0.1:2560 -CAfile
certs/ca.crt 2>&1 | grep -E "web.crt|Reason"
```

```
error 23 at 0 depth lookup: certificate revoked
certs/web.crt: revoked
    Reason: keyCompromise
```

8. Compare self-signed and third-party certificates

Create a quick self-signed certificate (no CA vouches for it) and compare it with the third-party roots shipped in the trust store, which is why public websites buy or obtain certificates from public CAs such as Let's Encrypt.

```
cd /root/ca
openssl req -x509 -newkey rsa:2048 -nodes -keyout /root/ca/private/selfsigned.key -days 30
-subj "/CN=selfsigned.lab.local" -out certs/selfsigned.crt 2>/dev/null
openssl x509 -in certs/selfsigned.crt -noout -subject -issuer
openssl verify certs/selfsigned.crt 2>&1 | tail -2
echo "trusted third-party roots in this OS: $(grep -c 'BEGIN CERTIFICATE'
/etc/ssl/certs/ca-certificates.crt)"
openssl crl2pkcs7 -nocrl -certfile /etc/ssl/certs/ca-certificates.crt | openssl pkcs7 -
print_certs -noout | grep -E "^subject=.*(ISRG Root X1|DigiCert Global Root G2)"
```

Test it

```
cd /root/ca
openssl x509 -in certs/web.crt -noout -ext subjectAltName | grep -c '\*.lab.local'
grep -c '^R' index.txt
openssl crl -in crl/lab.crl -noout -text | grep -c "Serial Number: 1000"
openssl ocsp -issuer certs/ca.crt -cert certs/web.crt -url http://127.0.0.1:2560 -CAfile
certs/ca.crt 2>/dev/null | grep -c "revoked"
```

Expected: `1` (wildcard SAN present), `1` (one revoked entry in the CA database), `1` (serial 1000 on the CRL) and `1` (OCSP reports the certificate as revoked).

Troubleshooting

Symptom	Fix
`openssl ca` fails with `There is already a certificate for /CN=*.lab.local`	`unique_subject = no` must be in `openssl.cnf`; if you ran the lab twice, also check that `index.txt.attr` contains `unique_subject = no`.
OCSP client prints `Error connecting BIO`	The responder is not running. Check `ss -tlnp | grep 2560` and `cat /root/ca/ocsp.log`, then rerun the `nohup openssl ocsp ...` line.
OCSP still says `good` after revoking	The responder read `index.txt` at start-up. Restart it as shown in step 7 so it loads the updated database.

Challenge

Build a proper **two-tier hierarchy**: create an intermediate CA signed by the root (with `pathlen:0`), issue a new server certificate from the intermediate, and verify it with `openssl verify -CAfile certs/ca.crt -untrusted intermediate.crt server.crt`. Explain why public CAs keep the root offline and issue from intermediates.

Reflection

A user reports a browser warning on an internal site that uses your lab CA. Give two different fixes, one using a self-signed root of trust and one using a third-party CA, and the trade-off between them.

Exam tips

- The CSR carries the **public key**; the **private key stays with the requester**. The CA signs with **its** private key.
- **CRL** = periodically published list; **OCSP** = real-time single-certificate query; **OCSP stapling** = the server attaches the signed response.
- **Wildcard** covers one level of subdomains (`*.lab.local` covers `web.lab.local`, not `a.b.lab.local`); **SAN** lists multiple names.
- A **self-signed** certificate is its own root of trust; **third-party** CA roots come pre-installed in trust stores.

References

- RFC 5280 X.509 PKI Certificate and CRL Profile — https://www.rfc-editor.org/rfc/rfc5280
- RFC 6960 Online Certificate Status Protocol (OCSP) — https://www.rfc-editor.org/rfc/rfc6960
- Let's Encrypt — how it works — https://letsencrypt.org/how-it-works/
- crt.sh Certificate Transparency search — https://crt.sh/
